Od dat k predikci: Trénink, validace a hodnocení prvního modelu

Kapitola 1: Rekapitulace a úvod do světa modelování

Most z minulé lekce: Jsme na startovní čáře

Vítejte zpět, kolegyně a kolegové. V minulé přednášce jsme se ponořili do nezbytné, i když méně okouzlující, fáze každého projektu strojového učení – přípravy dat. Prošli jsme si procesem čištění, škálování a především jsme se vypořádali s "prokletím dimenzionality" pomocí Analýzy hlavních komponent (PCA).
Na konci této cesty nám v rukou zůstal dokonalý artefakt: čistý, škálovaný a dimenzionálně redukovaný dataset, pravděpodobně ve formě Pandas DataFrame. Každý řádek v této tabulce reprezentuje jeden unikátní materiál a každý sloupec představuje jeden z jeho numerických deskriptorů – jeho digitální otisk prstu. V odděleném sloupci pak máme cílovou vlastnost, například mez pevnosti, kterou se chystáme predikovat. Veškerá přípravná práce je za námi. Stojíme na startovní čáře a před námi leží vzrušující svět prediktivního modelování. Jsme připraveni naučit počítač "myslet" jako materiálový vědec.

Největší nepřítel: Přeučení (Overfitting)

Než se ale pustíme do tréninku, musíme si promluvit o největším a nejzákeřnějším nepříteli celé datové vědy. Zeptám se vás: Co kdybych vám řekl, že jsem vytvořil model, který má na datech, která jsme mu ukázali, 100% úspěšnost? Dokáže dokonale reprodukovat výsledky pro všechny materiály v naší trénovací sadě. Je to skvělý model? Intuitivní odpověď je "ano", ale správná odpověď je téměř vždy "rozhodně ne". Takový model je naopak extrémně podezřelý a v praxi pravděpodobně k ničemu. Tento paradoxní jev se nazývá **přeučení** (overfitting).

Pro jeho pochopení použijme analogii, která vám bude blízká: student, který se učí na zkoušku.

- **Špatný student (Přeučený model):** Tento student dostane sadu 100 starých testových otázek i se správnými odpověďmi. Místo toho, aby se snažil pochopit principy a zákonitosti, které za řešením stojí, se všechny otázky a odpovědi naučí nazpaměť. Na zkoušce, která by obsahovala přesně těchto 100 otázek, by samozřejmě dosáhl 100% úspěšnosti. Pokud se ale objeví byť jen jediná nová otázka, která testuje stejný princip, ale je formulována trochu jinak, student naprosto selže. Nenaučil se totiž zobecňovat, naučil se pouze memorovat.
- **Dobrý student (Dobře generalizující model):** Tento student použije stejných 100 otázek k tomu, aby pochopil základní fyzikální a chemické principy. Jeho cílem není zapamatovat si konkrétní odpovědi, ale naučit se *metodu*, jak k nim dospět. Na zkoušce možná nezíská

100 %, třeba udělá chybu ve výpočtu a dostane 95 %. Jeho klíčovou schopností je ale to, že dokáže správně odpovědět i na zcela nové, dosud neviděné otázky, protože pochopil podstatu problému.

Přeučený model se chová přesně jako ten špatný student. Místo aby se naučil skutečný, zobecnitelný fyzikální vztah mezi deskriptory materiálu a jeho cílovou vlastností (signál), naučí se nazpaměť všechny náhodné šumy, odchylky měření a specifika konkrétních vzorků v trénovacích datech (šum). Takový model má vysokou varianci – jeho predikce jsou extrémně citlivé na drobná specifika trénovacích dat. Výsledkem je model, který exceluje na datech, která už viděl, ale je naprosto nepoužitelný pro predikci vlastností nového, reálného materiálu, se kterým se setká v laboratoři. Naším cílem tedy není vytvořit model, který si dokonale pamatuje minulost, ale takový, který dokáže spolehlivě předpovídat budoucnost.

Přeučení ve skutečnosti není jen technickou chybou, ale selháním v aplikaci samotné vědecké metody. Cílem vědy je vytvářet obecně platné modely a teorie, které nejen vysvětlují minulá pozorování, ale především dokáží predikovat výsledky nových, budoucích experimentů. Model, který se přeučil, je ekvivalentem "teorie", která dokonale popisuje pouze ty experimenty, ze kterých byla odvozena, ale selhává v predikci jakéhokoli nového experimentu. Takový model nedokázal oddělit fundamentální, přenositelný princip od náhodných chyb a specifik měření. Boj proti přeučení tedy není jen o ladění algoritmů; je to o zajištění, že náš datově řízený přístup skutečně objevuje smysluplné, fyzikálně podložené vztahy, a ne jen náhodné korelace.

Kapitola 2: Základní kámen důvěry – Validace modelu

Zlaté pravidlo datové vědy

Jak se tedy můžeme proti přeučení bránit? Jak si můžeme vybudovat důvěru v náš model a ověřit jeho schopnost zobecňovat? Vše stojí na jednom zlatém, neporušitelném pravidlu:
Výkon modelu se nikdy, za žádných okolností, nehodnotí na datech, která model viděl během trénování.
Porušit toto pravidlo je jako dát studentovi na zkoušku přesně ty otázky, které si předem připravil s řešením. Výsledné skóre nám neřekne absolutně nic o jeho skutečných znalostech a schopnosti řešit nové problémy.

První řešení: Rozdělení na trénovací a testovací sadu (Train/Test Split)

Nejzákladnější metodou, jak dodržet zlaté pravidlo, je procedura zvaná Train/Test Split. Princip je jednoduchý: vezmeme celý náš připravený dataset a náhodně ho rozdělíme na dvě části, typicky v poměru 80 % ku 20 %.

- **Trénovací sada (Training Set - 80 %):** Tuto větší část dat ukážeme modelu. Na ní se model "učí" – hledá vztahy, optimalizuje své vnitřní parametry. Je to ekvivalent studijních materiálů pro našeho studenta.
- **Testovací sada (Test Set - 20 %):** Tuto menší část dat si pečlivě uschováme a model ji

během tréninku nikdy neuvidí. Po dokončení tréninku ji použijeme k jedinému, finálnímu a nezávislému ohodnocení výkonu. Je to ekvivalent ostré zkoušky.

Jak ale zajistit, aby rozdělení bylo spravedlivé? Zde existují dva hlavní přístupy.

1. **Náhodný výběr (Random Sampling)**
Nejjednodušší metoda je data jednoduše zamíchat a náhodně rozdělit. Je to jako zamíchat balíček karet a rozdat je na dvě hromádky. Každý datový bod má stejnou šanci, že skončí v trénovací nebo testovací sadě. Tím se zabrání systematickému zkreslení (např. kdyby data byla seřazena od nejlevnějších slitin po nejdražší). V knihovně scikit-learn je toto chování výchozí pro funkci **train_test_split**.

2. Stratifikovaný výběr (Stratified Sampling)
Náhodný výběr má však riziko, zmíněné níže – u malých nebo nevyvážených datasetů můžeme mít smůlu. Řešením je stratifikovaný výběr, který je mnohem robustnější. Principem je zachovat v trénovací i testovací sadě stejný poměr (distribuci) nějaké klíčové vlastnosti, jako má celý dataset. Obvykle se stratifikuje podle cílové proměnné (toho, co se snažíme předpovědět).

- *Příklad u klasifikace:* Pokud máme v datech 20 % vzorků vadných a 80 % v pořádku, stratifikace zajistí, že i v testovací sadě bude přesně 20 % vadných a 80 % v pořádku. Tím se vyhneme situaci, kdy by v testu byly náhodou jen ty "jednoduché" případy.
- *Příklad u regrese:* Pokud predikujeme cenu slitiny, můžeme si ceny rozdělit do kategorií (např. levné, střední, drahé) a stratifikace nám pohlídá, aby v testovací sadě byl od každé kategorie reprezentativní vzorek.

Stratifikovaný výběr je téměř vždy lepší volbou a v scikit-learn ho lze jednoduše zapnout pomocí parametru **stratify**.

Problém malých dat: Proč jednoduché rozdělení nestačí

Tento přístup funguje skvěle pro velké datasety. V materiálové vědě ale často pracujeme s malými, draze získanými daty – desítkami, v lepším případě stovkami experimentálních bodů. A zde narážíme na kritickou slabinu jednoduchého rozdělení.
Představte si, že máme dataset s pouhými 50 vzorky slitin. Rozdělení 80:20 nám dá 40 vzorků pro trénink a jen 10 pro testování. Výsledná metrika výkonu, například průměrná chyba, bude vypočítána pouze z těchto 10 bodů. Takový odhad je statisticky velmi nestabilní a nespolehlivý. Navíc hraje obrovskou roli náhoda při samotném rozdělení. Co když se do testovací sady náhodou dostanou jen "jednoduché" slitiny s dobře predikovatelnými vlastnostmi? Náš model bude vypadat skvěle a my získáme falešně optimistický výsledek. A naopak, co když se tam dostanou jen ty nejexotičtější a nejproblematičtější vzorky? Výkon bude vypadat katastrofálně a my získáme falešně pesimistický odhad. Výsledná metrika bude mít vysokou varianci v závislosti na konkrétním náhodném rozdělení, což z ní činí nedůvěryhodný ukazatel.

Zlatý standard: Křížová validace (k-Fold Cross-Validation)

Robustním a v praxi nejpoužívanějším řešením tohoto problému je **křížová validace (k-Fold Cross-Validation)**. Je to chytřejší způsob, jak využít naše cenná data na maximum.

Představme si proces pro k=5:

1. Nejprve si odložíme stranou malou finální testovací sadu (např. 15 %), které se po celou dobu nedotkneme. Tato sada slouží jako zcela nezávislý arbitr pro finální ověření výkonu modelu poté, co dokončíme celý proces tréninku a ladění pomocí křížové validace. Simuluje reálné nasazení modelu na data, která nikdy předtím neviděl. Zbytek (85 %) tvoří naše data pro trénink a validaci.
2. Těchto 85 % dat náhodně rozdělíme na 5 stejně velkých "balíčků" neboli **foldů**. I zde platí, že pro nejlepší výsledky je vhodné použít stratifikované rozdělení, aby každý "balíček" měl podobnou distribuci dat jako celek.
3. **Iterace 1:** Trénujeme model na balíčcích 1, 2, 3 a 4. Výkon hodnotíme (validujeme) na balíčku 5. Uložíme si výsledek (např. hodnotu R^2 - koeficient determinace, viz dále).
4. **Iterace 2:** Zahodíme předchozí model. Trénujeme zcela nový model na balíčcích 1, 2, 3 a 5. Výkon hodnotíme na balíčku 4. Uložíme si výsledek.
5. Tento proces opakujeme celkem 5krát. Pokaždé použijeme jiný balíček pro validaci a zbylé 4 pro trénink.

Výsledkem není jedno číslo, ale 5 různých odhadů výkonu. Finální metrika výkonu je pak průměr těchto 5 hodnot.

Výhody tohoto přístupu jsou obrovské:

- Každý datový bod je právě jednou použit ve validační sadě, takže hodnotíme na celém datasetu.
- Využíváme data mnohem efektivněji, což je pro malé datasety klíčové.
- Výsledný průměrný výkon je mnohem stabilnější a spolehlivější odhad toho, jak se model bude chovat na skutečně nových, neviděných datech.

Křížová validace nám však dává víc než jen lepší průměr. Poskytuje nám celou *distribuci* odhadů výkonu – v našem případě pět různých hodnot R^2. Z této distribuce můžeme spočítat nejen průměr, ale i směrodatnou odchylku. A právě tato směrodatná odchylka je nesmírně cennou metrikou *stability* našeho modelu. Pokud je směrodatná odchylka vysoká (např. v jednom foldu máme R^2 0.9 a v druhém jen 0.5), je to varovný signál. Znamená to, že výkon modelu dramaticky závisí na tom, na které konkrétní podmnožině dat se trénuje. Taková citlivost na malé změny v trénovacích datech je charakteristickým znakem modelu s vysokou variancí, což je klíčový příznak přeučení. Naším cílem tedy není jen maximalizovat průměrný výkon, ale také zajistit, aby byl tento výkon konzistentní napříč různými podmnožinami dat, tedy minimalizovat jeho směrodatnou odchylku.

Kritérium	Train/Test Split	k-Fold Cross-Validation
Princip	Jedno rozdělení na trénovací a testovací sadu.	*k* iterací, kde se střídají trénovací a validační sady.
Robustnost odhadu	Nízká (výsledek silně závisí na náhodném rozdělení).	Vysoká (výsledek je průměr z *k* odhadů, menší variance).
Využití dat	Neefektivní (část dat se nikdy	Velmi efektivní (každý bod je

Kritérium	Train/Test Split	k-Fold Cross-Validation
	nepoužije pro trénink).	použit pro trénink i validaci).
Výpočetní náročnost	Nízká (model se trénuje pouze jednou).	Vyšší (model se trénuje *k*-krát).
Vhodnost pro malé datasety	Nevhodné (testovací sada je příliš malá, odhad je nestabilní).	**Ideální** (maximalizuje využití omezených dat).

Kapitola 3: Náš první model a jak měřit jeho úspěch

Představení prvního modelu: Lineární regrese

Nyní, když máme robustní strategii pro hodnocení, můžeme konečně natrénovat náš první model. Začneme s jedním z nejjednodušších, nejrychlejších a hlavně nejlépe interpretovatelných algoritmů: **Lineární regresí**.
Intuitivně je cílem lineární regrese proložit našimi datovými body přímku (v případě jednoho deskriptoru) nebo nadrovinu (v případě více deskriptorů), která co nejlépe popisuje vztah mezi deskriptory (nezávislé proměnné, X) a cílovou vlastností (závislá proměnná, Y). Matematicky se model snaží najít takové koeficienty (váhy) w_i a absolutní člen (bias) b v rovnici: Y_predik = w_1 · X_1 + w_2 · X_2 + … + w_n · X_n + b, aby se minimalizovala chyba mezi predikovanými hodnotami Y_predik a skutečnými hodnotami Y. Dělá to typicky pomocí metody nejmenších čtverců, která minimalizuje součet čtverců rozdílů (reziduí). Naučené hodnoty w_i a b jsou **parametry** modelu – jsou to hodnoty, které se model učí přímo z dat.

Metriky úspěchu pro regresi

Jak ale kvantitativně posoudit, zda je tato "nejlepší přímka" skutečně dobrá? K tomu slouží hodnotící metriky. Pro regresní úlohy, kde predikujeme spojitou hodnotu, jsou dvě nejdůležitější:

- **Střední absolutní chyba (Mean Absolute Error, MAE):**
  - **Definice:** MAE je průměrná absolutní hodnota rozdílu mezi predikovanou a skutečnou hodnotou pro všechny vzorky v testovací sadě. Vzorec je: MAE = 1/n · $\sum_{i=1}^{n}$ |Y_i - Y_pred,i|.
  - **Praktická interpretace:** Toto je nejdůležitější. MAE je metrika, která má stejné jednotky jako cílová veličina, což ji činí extrémně intuitivní. Pokud predikujete mez pevnosti v MPa a vaše MAE je 15, znamená to, že vaše predikce se v průměru mýlí o 15 MPa, ať už nahoru nebo dolů. Dává vám to přímou, srozumitelnou představu o velikosti chyby vašeho modelu v reálném světě.

- **Koeficient determinace (R^2):**
  - **Definice:** R^2 udává, jaký podíl variability (rozptylu) v cílové proměnné je vysvětlitelný naším modelem.
  - **Praktická interpretace:** R^2 srovnává náš model s tím nejhloupějším možným

modelem – takovým, který by pro každý materiál predikoval jednoduše průměrnou hodnotu cílové vlastnosti ze všech trénovacích dat.

- R^2 = 0 znamená, že náš model není o nic lepší než tento naivní průměr.
- R^2 = 1 znamená, že náš model dokonale vysvětluje všechna data bez chyby.
- R^2 = 0.85 znamená, že 85 % variability v mezi pevnosti našich slitin je vysvětleno deskriptory, které jsme do modelu vložili. Zbylých 15 % je způsobeno šumem nebo faktory, které náš model nezachycuje. Hodnota R^2 může být i záporná, což znamená, že model je ještě horší než pouhé hádání průměru.

Kapitola 4: Profesionální pracovní postup – Síla Scikit-learn Pipelines

Tichý zabiják: Únik dat (Data Leakage)

I když poctivě používáme křížovou validaci, stále můžeme udělat zákeřnou chybu, která znehodnotí naše výsledky. Tento problém se nazývá únik dat (Data Leakage) a je jedním z nejčastějších a nejhůře odhalitelných prohřešků v datové vědě.

Představte si, že jako součást přípravy dat potřebujeme provést nějakou úpravu napříč celým datasetem. Typickým příkladem je **škálování**, kdy všechny sloupce převedeme na společné měřítko (např. aby hodnoty byly mezi 0 a 1, nebo měly průměr 0 a rozptyl 1).

Naivní, ale velmi častý a **špatný postup** by vypadal takto:

1. Načteme kompletní dataset.
2. Provedeme škálování na **všech datech najednou**. K tomu si musíme spočítat určité parametry, například průměr a směrodatnou odchylku z celého datasetu.
3. Až poté takto upravená data rozdělíme na trénovací a testovací sadu.

Kde je ten zákeřný problém?

Problém je v tom, že při výpočtu průměru a směrodatné odchylky jsme použili i data, která později skončila v testovací sadě. Informace o distribuci testovacích dat (jejich průměr a rozptyl) tak **"prosákla"** do fáze přípravy. Trénovací data byla upravena s vědomím toho, jak vypadají data testovací.

Je to jemné, ale zásadní porušení zlatého pravidla – model, a dokonce ani přípravná fáze, nesmí mít o testovacích datech vůbec žádné informace. Tento únik vede k **uměle navýšeným, příliš optimistickým výsledkům**. Model dostal nápovědu a na testovacích datech si pak vede lépe, než by si vedl na skutečně nových, neviděných datech v praxi.

Správný postup je následující:

1. **Nejprve a ze všeho nejdříve** rozdělíme data na trénovací a testovací sadu. To je vždy první krok.
2. Vezmeme **pouze trénovací sadu** a na ní se "naučíme" parametry pro naši úpravu (např. spočítáme průměr a rozptyl jen z trénovacích dat).
3. Tento naučený transformační postup pak **aplikujeme zvlášť** na trénovací sadu a zvlášť na testovací sadu.

Tímto způsobem zůstanou testovací data skutečně nedotčená a budou dokonale simulovat nová data z reálného světa. Finální hodnocení modelu pak bude objektivní a pravdivé.

Elegantní řešení: Scikit-learn Pipelines

Hlídat si ručně správné pořadí všech přípravných a modelovacích kroků v každém kole křížové validace by bylo pracné a náchylné k chybám. Naštěstí existuje elegantní a profesionální řešení: koncept zvaný **Pipeline** (v překladu „potrubí“ nebo „výrobní linka“).

Představte si Pipeline jako jeden balíček, který v sobě uzamkne celý „recept“ na váš model: od prvotních úprav dat (jako je škálování) až po samotný predikční algoritmus. Všechny kroky jsou v něm pevně seřazeny. Navenek se pak celý tento komplexní proces chová jako jeden jediný, jednoduchý „super-model“.

Kouzlo spočívá v tom, že když tento kompletní „recept“ předáme nástroji pro křížovou validaci, ten se postará o správné provedení zcela automaticky. Pro každé jednotlivé kolo křížové validace sám zajistí, že:

1. Vezme si **pouze trénovací část dat** daného kola.
2. Na ní **„naučí“ všechny přípravné kroky** (například zjistí, jaký je průměr a rozptyl potřebný pro škálování).
3. Tuto naučenou přípravu pak **aplikuje jak na trénovací, tak na validační část dat**.
4. Nakonec na takto čerstvě připravených datech **natrénuje a ohodnotí samotný model**.

Únik dat je tak z principu vyloučen. Kromě této zásadní výhody v oblasti spolehlivosti je celý postup mnohem čistší, přehlednější a snadno opakovatelný.

Použití Pipeline je více než jen technický trik. Je to praktická ukázka principu **zapouzdření**, známého z programování. Místo série oddělených kroků a dat, které musíme manuálně spravovat, vytvoříme jediný, soběstačný objekt, který reprezentuje celý náš modelovací proces. Tento objekt můžeme uložit, sdílet a znovu použít s jistotou, že kdokoli ho spustí, provede přesně stejné kroky ve správném pořadí. To dramaticky zvyšuje reprodukovatelnost a důvěryhodnost vědeckých výsledků.

Kapitola 5: Od dobrého modelu k nejlepšímu – Optimalizace hyperparametrů

Dva typy "knoflíků": Parametry vs. Hyperparametry

Zatím jsme mluvili o tom, že se model "učí" parametry z dat. Ale téměř každý model má i další sadu "knoflíků" a "nastavení", které musíme zvolit my, datoví vědci. Je klíčové rozumět rozdílu mezi těmito dvěma typy.

- **Parametry modelu:** Jsou to vnitřní proměnné modelu, jejichž hodnoty se model *učí* z dat během tréninku. Jsou *výstupem* trénovacího procesu. U naší lineární regrese jsou to naučené koeficienty (váhy) w_i a absolutní člen b.
- **Hyperparametry modelu:** Jsou to vnější "nastavení" nebo "konfigurace" algoritmu, které musíme zvolit *před* zahájením tréninku. Nejsou učená z dat, ale řídí samotný proces učení. Jsou *vstupem* do trénovacího procesu. Pro jednoduchou lineární regresi jich mnoho není, ale u pokročilejších modelů, které uvidíme příště, to bude například počet stromů v Random Forest nebo síla regularizace.

Aspekt	Parametry modelu	Hyperparametry modelu
Definice	Vnitřní proměnné modelu, jejichž hodnoty se odhadují z dat.	Vnější konfigurační proměnné, které se nastavují před tréninkem.
Zdroj	Učí se z trénovacích dat.	Nastavuje je uživatel/datový vědec.
Role	Jsou podstatou samotného "naučeného" modelu; používají se k predikcím.	Řídí proces učení a komplexitu modelu.
Příklad (Lineární regrese)	Koeficienty (váhy w) a absolutní člen (bias b).	Zda použít regularizaci a jak silnou (např. v Ridge regresi).
Jak se hledají?	Optimalizačním algoritmem (např. metoda nejmenších čtverců).	Laděním (Hyperparameter Tuning), např. pomocí Grid Search.

Systematické ladění: Grid Search a Random Search

Jak tedy najít nejlepší hodnoty hyperparametrů? Zkoušet je ručně metodou pokus-omyl je neefektivní. Proto existují systematické techniky.

- **Grid Search (Hledání v mřížce):**
  - **Princip:** Pro každý hyperparametr, který chceme ladit, definujeme seznam (nebo "mřížku") hodnot, které chceme vyzkoušet. Algoritmus pak systematicky vyzkouší *každou možnou kombinaci* z této mřížky.
  - **Analogie:** Je to jako zkoušet všechny kombinace na kombinačním zámku. Je to vyčerpávající, ale máme zaručeno, že najdeme nejlepší kombinaci *v rámci námi definované mřížky*.
  - **Nevýhoda:** Výpočetně velmi náročné, pokud máme mnoho hyperparametrů nebo mnoho hodnot k vyzkoušení. Počet kombinací roste exponenciálně.

- **Random Search (Náhodné hledání):**
    - **Princip:** Místo definování pevných hodnot definujeme pro každý hyperparametr distribuci (např. "vyber náhodné celé číslo mezi 10 a 100"). Algoritmus pak provede předem daný počet náhodných "výstřelů" do tohoto prostoru hyperparametrů.
    - **Výhoda:** Často je překvapivě mnohem efektivnější. Studie ukázaly, že často jen několik málo hyperparametrů má skutečně zásadní vliv na výkon modelu. Random Search má větší šanci rychle najít dobrou kombinaci, protože neplýtvá časem na zkoušení mnoha variant nedůležitých hyperparametrů.

Spojení všeho dohromady: GridSearchCV

Scikit-learn nabízí nástroj GridSearchCV, který geniálně kombinuje ladění hyperparametrů s křížovou validací, a to vše v dokonalé synergii s Pipeline.

- Masivní, ale metodologicky čistý proces, kde pro *každou jednotlivou kombinaci* hyperparametrů v mřížce se provede kompletní *k-fold cross-validation* a vypočítá se průměrný výkon pro danou kombinaci napříč všemi foldy.
- Po vyzkoušení všech kombinací se najde ta, která měla nejlepší průměrný výkon.
- Nakonec se automaticky natrénuje nový model s touto nejlepší kombinací hyperparametrů na *celém* trénovacím datasetu. Tento finální, optimalizovaný model je pak výsledkem celého procesu, připravený k použití.

Toto je nejrobustnější a v praxi nejpoužívanější způsob, jak vybrat a naladit model, který bude spolehlivě generalizovat.

Kapitola 6: Závěr a výhled na příští týden

Shrnutí univerzálního pracovního postupu

Dnes jsme si poskládali kompletní, robustní a profesionální pracovní postup pro řešení regresních úloh ve strojovém učení. Tento postup je univerzální a měl by se stát vaší mantrou, protože vás chrání před největšími nástrahami: přeučením, únikem dat a chybným výběrem hyperparametrů.

Pojďme si ho zrekapitulovat:

1. **Příprava dat:** Začínáme s čistým, featurizovaným DataFrame (výsledek přednášek 2 a 3).
2. **Finální rozdělení:** Pomocí train_test_split oddělíme malou finální testovací sadu (např. 15-20 %). Tuto sadu "zamkneme do trezoru" a až do úplného konce se jí nedotkneme. Vše ostatní se děje na zbylé, větší trénovací sadě.
3. **Definice Pipeline:** Zapouzdříme všechny kroky předzpracování (např. StandardScaler, PCA) a samotný model (např. LinearRegression) do jednoho Pipeline objektu.
4. **Definice mřížky hyperparametrů:** Specifikujeme, které hyperparametry a jejich hodnoty chceme pro jednotlivé kroky pipeline vyzkoušet.
5. GridSearchCV spustí celý automatizovaný proces hledání nejlepší kombinace hyperparametrů pomocí křížové validace na trénovacích datech.

6. **Finální hodnocení:** Po dokončení tréninku "odemkneme trezor". Vezmeme nejlepší nalezený model a provedeme finální, jediné a nezávislé ohodnocení jeho výkonu na dosud nepoužité testovací sadě. Toto číslo je náš nejlepší odhad toho, jak se model bude chovat v reálném světě.

Navnadění na příští lekci: Za hranice linearity

Dnes jsme udělali obrovský kus práce a postavili jsme si první, funkční a robustně vyhodnocený model. Lineární regrese je skvělý start, ale má jedno zásadní omezení – předpokládá, že vztah mezi deskriptory a cílovou vlastností je... no, lineární. Ale co když tomu tak není? Co když je vztah mezi složením slitiny a její tažností složitě nelineární?
Příští týden opustíme svět přímek a rovin a podíváme se na mnohem flexibilnější a často i výkonnější modely, které si s nelinearitou hravě poradí.

- **Rozhodovací stromy (Decision Trees):** Představíme si intuitivní modely, které se učí sérii jednoduchých "if-then" pravidel, podobně jako se rozhoduje člověk. Jsou snadno vizualizovatelné a interpretovatelné.
- **Ansámblové metody:** Ukážeme si sílu kolektivní inteligence. Zjistíme, jak můžeme zkombinovat stovky relativně "slabých" rozhodovacích stromů a vytvořit z nich extrémně silný a robustní predikční model, který patří ke špičce v klasickém strojovém učení.
    - **Random Forest:** Seznámíme se s "demokracií" stromů, kde každý strom hlasuje a finální predikce je výsledkem většinového rozhodnutí.
    - **Gradient Boosting:** Prozkoumáme "týmovou práci" stromů, kde se každý další strom učí z chyb toho předchozího a postupně tak zpřesňuje celkovou predikci.

Těmito nástroji se vybavíme na řešení skutečně komplexních problémů, které jsou v materiálové informatice na denním pořádku.

Citovaná díla

1. Overfitting and Underfitting in Machine Learning - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2020/02/underfitting-overfitting-best-fitting-machine-learning/ 2. What Is Overfitting in Machine Learning?Overfitting in ML Explained Simply with Real Examples | M005 | by Mehul Ligade | Towards AI, https://pub.towardsai.net/what-is-overfitting-in-machine-learning-ebdf40bb21b1 3. ML | Underfitting and Overfitting - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/ 4. What is Overfitting? - Overfitting in Machine Learning Explained - AWS - Updated 2025, https://aws.amazon.com/what-is/overfitting/ 5. What is Bias-Variance Tradeoff? | IBM, https://www.ibm.com/think/topics/bias-variance-tradeoff 6. Overfitting | Machine Learning - Google for Developers, https://developers.google.com/machine-learning/crash-course/overfitting/overfitting 7. Why is Model Evaluation Important in Machine Learning? - Comet, https://www.comet.com/site/blog/why-is-model-evaluation-important-in-machine-learning/ 8. 3.1. Cross-validation: evaluating estimator performance - Scikit-learn, https://scikit-learn.org/stable/modules/cross_validation.html 9. Model Evaluation Techniques in Machine Learning | by Fatmanurkutlu - Medium,

https://medium.com/@fatmanurkutlu1/model-evaluation-techniques-in-machine-learning-8cd88deb8655 10. Split Your Dataset With scikit-learn's train_test_split() - Real Python, https://realpython.com/train-test-split-python-data/ 11. Train Test Split: What it Means and How to Use It - Built In, https://builtin.com/data-science/train-test-split 12. Understanding Train-Test Split | Kaggle, https://www.kaggle.com/discussions/general/542494 13. Train-Test Split In Machine Learning | by Odeyemi Temitope | Medium, https://medium.com/@odeyemitemi23/train-test-split-in-machine-learning-cc0c02221e8b 14. train_test_split — scikit-learn 1.7.2 documentation, https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html 15. Is there a rule-of-thumb for how to divide a dataset into training and validation sets? [closed], https://stackoverflow.com/questions/13610074/is-there-a-rule-of-thumb-for-how-to-divide-a-dataset-into-training-and-validatio 16. Disadvantages of train-test split - machine learning - Stack Overflow, https://stackoverflow.com/questions/54904680/disadvantages-of-train-test-split 17. Is it wrong to compare models evaluated on different train/test splits? - Reddit, https://www.reddit.com/r/MLQuestions/comments/1fgtsfv/is_it_wrong_to_compare_models_evaluated_on/ 18. Cross Validation in Machine Learning - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/ 19. An Intuitive Guide to Understanding Cross-validation in Machine Learning - Medium, https://medium.com/learning-data/an-intuitive-guide-to-understanding-cross-validation-abd01bf5e879 20. Cross-Validation - MLU-Explain, https://mlu-explain.github.io/cross-validation/ 21. Cross-Validation in Machine Learning: How to Do It Right - neptune.ai, https://neptune.ai/blog/cross-validation-in-machine-learning-how-to-do-it-right 22. Cross-validation (statistics) - Wikipedia, https://en.wikipedia.org/wiki/Cross-validation_(statistics) 23. intuition - Cross-Validation in plain english?, https://stats.stackexchange.com/questions/1826/cross-validation-in-plain-english 24. Bias–variance tradeoff - Wikipedia, https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff 25. What Is Linear Regression? | IBM, https://www.ibm.com/think/topics/linear-regression 26. Regression: Definition, Analysis, Calculation, and Example - Investopedia, https://www.investopedia.com/terms/r/regression.asp 27. Linear Regression in Machine learning - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/ 28. Linear regression | Machine Learning - Google for Developers, https://developers.google.com/machine-learning/crash-course/linear-regression 29. Mean absolute error - Wikipedia, https://en.wikipedia.org/wiki/Mean_absolute_error 30. Mean Absolute Error In Machine Learning: What You Need To Know - Arize AI, https://arize.com/blog-course/mean-absolute-error-in-machine-learning-what-you-need-to-know/ 31. Understanding Mean Absolute Error (MAE) in Regression: A Practical Guide - Medium, https://medium.com/@m.waqar.ahmed/understanding-mean-absolute-error-mae-in-regression-a-practical-guide-26e80ebb97df 32. Mean Absolute Error Explained: Measuring Model Accuracy - DataCamp, https://www.datacamp.com/tutorial/mean-absolute-error 33. Coefficient of determination - Wikipedia, https://en.wikipedia.org/wiki/Coefficient_of_determination 34. How To Interpret R-squared in Regression Analysis - Statistics By Jim, https://statisticsbyjim.com/regression/interpret-r-squared-regression/ 35. Intuitive Explanation of R-squared - Data Science Stack Exchange, https://datascience.stackexchange.com/questions/45256/intuitive-explanation-of-r-squared 36. R-squared, Clearly Explained!!! - YouTube, https://www.youtube.com/watch?v=2AQKmw14mHM 37. 11. Common pitfalls and

recommended practices - Scikit-learn, https://scikit-learn.org/stable/common_pitfalls.html 38. What is Data Leakage in Machine Learning? - IBM, https://www.ibm.com/think/topics/data-leakage-machine-learning 39. How to Avoid Data Leakage When Performing Data Preparation - MachineLearningMastery.com, https://machinelearningmastery.com/data-preparation-without-data-leakage/ 40. Why Feature Scaling Should Be Done After Splitting Your Dataset into Training and Test Sets - DEV Community, https://dev.to/gervaisamoah/why-feature-scaling-should-be-done-after-splitting-your-dataset-into-training-and-test-sets-14ia 41. How to prevent data leakage in pandas & scikit-learn, https://www.dataschool.io/machine-learning-data-leakage/ 42. Data normalization before or after train-test split? - Data Science Stack Exchange, https://datascience.stackexchange.com/questions/54908/data-normalization-before-or-after-train-test-split 43. 7.1. Pipelines and composite estimators — scikit-learn 1.7.2 documentation, https://scikit-learn.org/stable/modules/compose.html 44. The Power of Scikit-learn Pipelines: Streamlining Your Machine Learning Workflow, https://aravindkolli.medium.com/the-power-of-scikit-learn-pipelines-streamlining-your-machine-learning-workflow-8f4cb2e8e1df 45. Python ML pipelines with Scikit-learn: A beginner's guide - SAS Blogs, https://blogs.sas.com/content/sgf/2025/05/23/python-ml-pipelines-with-scikit-learn/ 46. What is exactly sklearn.pipeline.Pipeline? - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/what-is-exactly-sklearnpipelinepipeline/ 47. Scikit-learn Pipelines Explained: Streamline and Optimize Your Machine Learning Processes | by Sahin Ahmed, Data Scientist | Medium, https://medium.com/@sahin.samia/scikit-learn-pipelines-explained-streamline-and-optimize-your-machine-learning-processes-f17b1beb86a4 48. Prevent Data Leakage in Cross-Validation - Medium, https://medium.com/@pacosun/split-smart-avoiding-data-leakage-in-cross-validation-894863a93553 49. Streamline Your Machine Learning Workflow with Scikit-learn Pipelines - KDnuggets, https://www.kdnuggets.com/streamline-your-machine-learning-workflow-with-scikit-learn-pipelines 50. The Power of Pipelines - MachineLearningMastery.com, https://machinelearningmastery.com/the-power-of-pipelines/ 51. Why You Should Use Scikit-Learn Pipelines | Towards Data Science, https://towardsdatascience.com/why-you-should-use-scikit-learn-pipelines-8754b4d1e375/ 52. Parameters and Hyperparameters in Machine Learning and Deep Learning, https://towardsdatascience.com/parameters-and-hyperparameters-aa609601a9ac/ 53. Difference Between Model Parameters VS HyperParameters - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/difference-between-model-parameters-vs-hyperparameters/ 54. What is the difference between model hyperparameters and model parameters?, https://datascience.stackexchange.com/questions/14187/what-is-the-difference-between-model-hyperparameters-and-model-parameters 55. Parameters vs hyperparameters in machine learning - YouTube, https://www.youtube.com/watch?v=V4AcLJ2cgmU 56. What is Hyperparameter Tuning? - AWS, https://aws.amazon.com/what-is/hyperparameter-tuning/ 57. What Is Hyperparameter Tuning? - IBM, https://www.ibm.com/think/topics/hyperparameter-tuning 58. Grid Search vs Random Search: Which One Should You Use? - Train in Data's Blog, https://www.blog.trainindata.com/grid-search-vs-random-search-which-one-should-you-use/ 59. Hyperparameter Tuning: Grid Search, Random Search, and Bayesian Optimization, https://keylabs.ai/blog/hyperparameter-tuning-grid-search-random-search-and-bayesian-optimiz

ation/ 60. Hyperparameter Tuning Showdown: Grid Search vs. Random Search — Which is the Ultimate Winner? | by Hestisholihah | Medium, https://medium.com/@hestisholihah01/hyperparameter-tuning-showdown-grid-search-vs-random-search-which-is-the-ultimate-winner-5927b322e54d 61. [D] A few questions on Hyperparameter search : r/MachineLearning - Reddit, https://www.reddit.com/r/MachineLearning/comments/5prv3m/d_a_few_questions_on_hyperparameter_search/ 62. Practical hyperparameter optimization: Random vs. grid search - Cross Validated - Stats StackExchange, https://stats.stackexchange.com/questions/160479/practical-hyperparameter-optimization-random-vs-grid-search 63. GridSearchCV — scikit-learn 1.7.2 documentation, https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html 64. 3.2. Tuning the hyper-parameters of an estimator - Scikit-learn, https://scikit-learn.org/stable/modules/grid_search.html 65. How does GridSearchCV use Cross Validation to produce a Model's Score?, https://datascience.stackexchange.com/questions/117596/how-does-gridsearchcv-use-cross-validation-to-produce-a-models-score 66. Does GridSearchCV perform cross-validation? - Stack Overflow, https://stackoverflow.com/questions/49160206/does-gridsearchcv-perform-cross-validation 67. Tuning Pipelines with GridSearchCV in scikit-learn - CodeSignal, https://codesignal.com/learn/courses/hypertuning-classical-models/lessons/tuning-pipelines-with-gridsearchcv-in-scikit-learn 68. Learn Data Science: A Simple Guide to Decision Trees : r/learndatascience - Reddit, https://www.reddit.com/r/learndatascience/comments/1kmwecb/learn_data_science_a_simple_guide_to_decision/ 69. Decision Trees Explained | Towards Data Science, https://towardsdatascience.com/decision-trees-explained-3ec41632ceb6/ 70. Decision Tree Algorithm - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2021/08/decision-tree-algorithm/ 71. What Is Random Forest? | IBM, https://www.ibm.com/think/topics/random-forest 72. Random Forest: A Complete Guide for Machine Learning | Built In, https://builtin.com/data-science/random-forest-algorithm 73. What is Gradient Boosting? - IBM, https://www.ibm.com/think/topics/gradient-boosting 74. A Guide to The Gradient Boosting Algorithm - DataCamp, https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm 75. Gradient Boosting Algorithm: A Complete Guide for Beginners, https://www.ccs.neu.edu/home/vip/teach/MLcourse/4_boosting/lecture_notes/boosting/Gradient Boosting_classifier.pdf